

Proofly Backend — Complete Guide

What was built, which file holds what, and how to use every API from the frontend.

Team: Vipul Kataria & Sorav Singh · Generated 18 Jul 2026 · Full JSON shapes live in [server/API.md](#)

1. What we built

Proofly = a GitHub-style proof-of-work profile for every profession. Core thesis: **we rank consistency, not claims**. The backend is Node/Express + MongoDB (Mongoose), stateless JWT auth via Passport, professions driven entirely by config.

Area	What it does
Authentication	Register, login (passport-local), JWT on every request (passport-jwt), change password, 7-day tokens, rate limiting (20 req / 10 min per IP on register+login).
Authorization	Layered middleware: requireAuth (valid token) -> requireRole (profession chosen) -> requireAdmin (isAdmin flag). Ownership enforced in every query (userId scoped).
Contributions	Daily work log per role metrics, 30-day backfill limit, no future dates, sanitized metrics, owner-scoped delete, optional evidence link.
Verification ladder	Rung 1 self_reported -> Rung 2 evidence (any http(s) proof link, at log time or attached later) -> Rung 3 imported (Excel import for Meta Ads, GitHub auto-sync for developers).
GitHub auto-sync	Developer connects a GitHub username; server pulls ~90 days of public commits/PRs as verified entries. Manual sync endpoint + automatic daily 03:00 cron. Idempotent.
Badges	9 config-driven achievements (streaks, active days, verified entries, score). Earned ones appear on the public profile automatically.
Insights	Week-vs-week trend, change %, best day, days-since-last-log, ready-made nudge strings for the dashboard.
Goals	Weekly target per metric (e.g. 20 calls/week), progress resets every Monday, pct precomputed for progress bars.
Streak freezes	2 per calendar month, cover a missed day from the last 7 — bridge the streak, add no score. Duolingo-style retention.
Public / showcase	Public profile (one call), embeddable SVG card (26-week heatmap, 1h cache), recruiter search with minStreak filter, per-role leaderboard.
Export	Downloadable self-contained HTML proof-of-work report (print -> PDF) + full JSON data takeout.
Support & site	About/Team content API, support tickets (guests welcome, spam-limited), users see their own tickets.
Admin	Overview counts, user list, ticket management. Admin is granted only via terminal script — no API can self-promote.
Email jobs	Daily 18:00 streak-at-risk reminder, Monday 08:00 digest. Off unless SMTP env vars are set; per-user opt-out.
Build check	npm run build imports all 29 modules to catch syntax/import errors before deploy (no real build needed — plain Node).

2. Which file has what code (server/src)

config/ — everything is data

File	Contents
config/roles.js	THE core file. 6 professions with metrics, weights, colors. Forms, heatmap, score, leaderboard all generate from this. sanitizeMetrics() whitelists+clamps input. Add a profession = add an entry, zero code.
config/badges.js	9 badge definitions (icon, label, desc, earned-check). badgesFor(summary) evaluates them.
config/site.js	About text + team (Vipul Kataria, Sorav Singh) + support topics. Served by GET /api/site.
config/passport.js	Two Passport strategies: 'local' (email+password w/ bcrypt) for login, 'jwt' (Bearer token -> load user) for every protected request.

models/ — Mongoose schemas

File	Contents
models/User.js	name, email, username (unique), passwordHash, role, headline, githubUsername, streakFreezes[dates], emailReminders, isAdmin.
models/Contribution.js	userId (ref User), role, date (yyyy-mm-dd string by design), metrics (Map), weightedTotal, note, evidenceUrl, verification enum, source enum. Indexed userId+date.
models/Goal.js	userId (ref User), metricKey, weeklyTarget. Unique index userId+metricKey — one goal per metric.
models/SupportTicket.js	userId (ref User, NULL for guests), name, email, topic, message, status open/closed.

middleware / services / jobs

File	Contents
middleware/auth.js	signToken (7d JWT), requireAuth + optionalAuth (both via passport-jwt), requireRole, requireAdmin.
services/dates.js	yyyy-mm-dd string utilities (UTC-noon math kills timezone bugs): isValidDateStr, addDays, diffDays, startOfWeek, isAllowedLogDate.
services/stats.js	dailyTotals (per user), totalsByUser (batch — ONE aggregation for leaderboard/search), computeLevels (percentile heatmap), computeStreaks (freeze-aware), computeScore, summaryFor.
services/svgCard.js	Builds the embeddable 640×196 SVG card: 26-week grid in role color, streak, score. Pure strings, no deps.
services/github.js	GitHub public-events fetch -> per-day commit/PR counts; syncGithubUser() wipes+reinserts github_sync rows (idempotent).
services/profileExport.js	Builds the self-contained HTML proof-of-work report (stats, card, badges, verification table, work log, print CSS).
services/mailer.js	Nodemailer wrapper. No SMTP env -> every send is a silent no-op.
services/excelParser.js	Meta Ads .xlsx/.csv template builder + parser (pre-existing).
jobs/scheduler.js	node-cron: 03:00 GitHub re-sync, 18:00 streak reminder, Mon 08:00 digest.
check.js	npm run build — imports every module, fails loud on broken code.
index.js	App wiring: env validation, CORS, passport init, route mounts, error handler, DB connect, startJobs().

routes/ — the API surface

File	Endpoints
routes/auth.js	POST /register, POST /login, GET /me, POST /change-password. Rate limiter lives here.
routes/users.js	PUT /role (locked once), PUT /profile, GET /streak-freezes, POST /streak-freeze.
routes/contributions.js	POST / (with evidenceUrl), GET /, PUT /:id/evidence, DELETE /:id.
routes/stats.js	GET /heatmap, /summary, /badges, /insights.
routes/goals.js	GET /, PUT /, DELETE /:metricKey.
routes/integrations.js	PUT /github, POST /github/sync, DELETE /github (developer-only).
routes/public.js	GET /search, GET /:username/card.svg, GET /:username — all public.
routes/leaderboard.js	GET /?role= — top 20 by streak (single batch aggregation).
routes/export.js	GET /profile (HTML report), GET /data (JSON takeout).
routes/support.js	POST / (public + optionalAuth, spam-limited), GET /mine.
routes/admin.js	GET /overview, /users, /tickets, PUT /tickets/:id — requireAdmin.
routes/importMetaAds.js	GET /template, POST /preview, POST /confirm (pre-existing Excel flow).

3. How to use it from the frontend

Base URL: `http://localhost:5001/api` (prod: `VITE_API_URL`). **Token:** login/register return `{token, user}` — send `Authorization: Bearer <token>` on every protected call; tokens last 7 days, on 401 log out. **Errors:** always `{error, field?}` — map field to the right input. **Dates:** always "yyyy-mm-dd" strings. **429:** rate-limited — show error, disable button briefly.

Session & profile

Endpoint	Frontend usage
POST <code>/auth/register</code> · <code>/auth/login</code>	Store token, route by <code>user.role</code> : null -> choose-role screen, else dashboard.
GET <code>/auth/me</code>	On app load to restore session. <code>user.isAdmin</code> -> show admin link; <code>user.githubUsername</code> -> GitHub connected state.
POST <code>/auth/change-password</code>	Settings page. Send <code>currentPassword</code> + <code>newPassword</code> ; 401 = wrong current password.
PUT <code>/users/role</code>	Choose-role screen. One-time — hide the option after it's set.
PUT <code>/users/profile</code>	Edit name/headline; <code>emailReminders:false</code> = the email opt-out toggle.

Daily loop (dashboard)

Endpoint	Frontend usage
GET <code>/roles</code>	On boot, cache it. Build the log form + labels + colors from it. Never hardcode metrics.
POST <code>/contributions</code>	Log form. Include <code>evidenceUrl</code> input ('link to proof — PR, campaign, design...'). With link -> entry saves as evidence (show the evidence badge).
PUT <code>/contributions/:id/evidence</code>	'Add proof' button on self-reported entries — upgrades them to evidence.
GET <code>/stats/summary</code>	Stat cards: streak, longest, active days, score, verified % (<code>verifiedCount/totalContributions</code>).
GET <code>/stats/heatmap</code>	<code>days[{date,total,level}]</code> — level 0-4 already computed, just paint the squares.
GET <code>/stats/insights</code>	This-week panel: <code>changePct</code> , <code>bestDay</code> , <code>nudges[]</code> are ready-made strings — render as banners.
GET <code>/stats/badges</code>	Achievements grid — earned vs locked (grey the locked ones).
GET/PUT/DELETE <code>/goals</code>	Weekly targets; response has <code>progress</code> + <code>pct</code> (capped 100) — draw bars directly.
POST <code>/users/streak-freeze</code>	When a gap threatens a streak, offer 'Use a freeze?' — GET <code>/users/streak-freezes</code> shows remaining.

Showcase (public pages — no token)

Endpoint	Frontend usage
GET <code>/public/:username</code>	Whole public profile in one call: profile, summary, earned badges, heatmap, recent (with <code>evidenceUrl</code> links). 404 -> not-found page.
GET <code>/public/:username/card.svg</code>	Show an 'Embed your card' snippet with copy button: <code></code> .
GET <code>/public/search?role=&q=&minStreak=</code>	Recruiter discovery page with filters — results ranked by streak.
GET <code>/leaderboard?role=</code>	Per-role leaderboard (role param required).

Power features

Endpoint	Frontend usage
PUT /integrations/github {username}	Developer settings: connect input. 404 = no such GitHub user.
POST /integrations/github/sync	'Sync now' button -> {synced: n} toast. Server also re-syncs daily at 03:00.
DELETE /integrations/github	Disconnect — warn: removes synced history.
GET /export/profile	'Download report' button. Fetch as blob (responseType:'blob'), createObjectURL, click an <a download>. Self-contained HTML; Ctrl+P -> clean PDF.
GET /export/data	'Download my data' (JSON) — same blob pattern.
GET /site	About + Team page content (Vipul & Sorav) + supportTopics for the form dropdown.
POST /support	Support form. Works logged-out; when logged in send the token — name/email auto-fill server-side.
GET /support/mine	'My tickets' list for logged-in users.
GET /admin/*	Admin pages — only render when user.isAdmin. Tickets can be closed via PUT /admin/tickets/:id.

Suggested integration order

1) /roles + /site on boot 2) auth flow + /auth/me 3) dashboard (summary + heatmap + insights in parallel) 4) log form with evidence field 5) public profile + card embed + leaderboard + search 6) goals, freeze, GitHub, export 7) support + admin.

Commands & environment

Command / var	Purpose
npm run dev	Start API with watch on :5001.
npm run build	Import-check all modules — run before commit and in Render's build command (npm install && npm run build).
npm run seed	8 demo users with history (arjun / rahul / priya... password demo1234).
npm run make-admin -- email	Grant admin to an existing account (only way to become admin).
MONGO_URI, JWT_SECRET	Required — server refuses to boot without them.
CLIENT_URL, PORT	CORS origin for prod; port (default 5001).
SMTP_HOST/PORT/USER/PASS, EMAIL_FROM	Optional — enables reminder + digest emails.